

ネットワーク系演習 I — システムソフトウェア演習 — 最終レポート

名古屋工業大学 創造工学教育課程 情報ネットワーク工学分野

学籍番号：36719042

氏名：渡邊 寿紀

2026 年 6 月 22 日

【開発環境】

最終成果物の作成および動作検証を行った環境を以下に示す。

- 使用 PC：MacBook Air (M3, 2024)
- OS：macOS26
- C コンパイラ：gcc (GCC) 11.x
- 開発ツール・ライブラリ：GNU Make, 字句解析ライブラリ libics.a

(1) 最終的に作成した記号表の構造と操作関数の仕様

1. 記号表の構造とコンパイラにおける役割

記号表は、原始プログラム中に出現する変数や手続きなどの識別子 (IDENTIFIER) を管理し、その属性や割当番地、スコープを保持するための極めて重要なデータ構造である。本コンパイラの実装では、ヘッダファイル `symtab.h` およびソースファイル `symtab.c` に独立したモジュールとして管理機構を作成している。

現在割り当てられている記号表は、最大登録数 `MAXSYMS` (32 個) を上限とする 1 次元の構造体配列である。識別子名となる文字列 `char name[MAXIDLEN+1]` をキーとして持って、配列のインデックスそのものが、仮想マシン `sr` における大域変数の固定データメモリ番地に対応するマッピング構造をつかっている。また、数式評価の時に動的に生成される一時確保用の変数もこの記号表へ逐次追加登録して、一元的に番地管理がされる。

さらに、開発が間に合わなかった機能である「手続き (PROCEDURE) 処理」への対応を見据え、

手続き名とその開始ラベルの対応関係を保持する配列 `proc_label` を記号表配列と並列にマッピングさせるなど取り組んでいる。

2. 記号表のメモリマッピング概念図

コンパイル時における識別子とデータ領域番地、および一次変数の記号表上での配置構造を図 1 に示す。

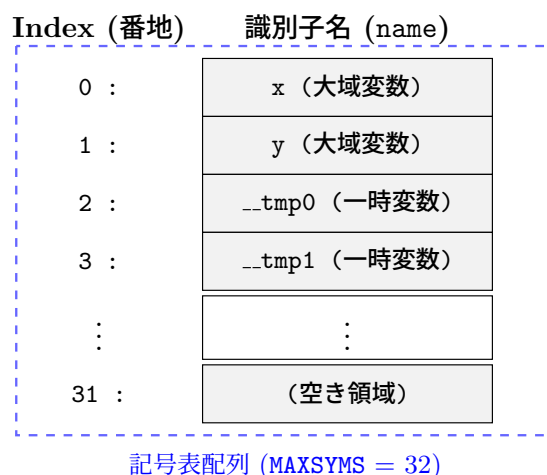


図 1 記号表の管理構造とデータアドレスの割り当て概念図

3. 操作関数の仕様とアルゴリズム

記号表の操作は、以下の 3 つの関数によって定義されている。

- `int sym_install(char *name);`
引数として渡された識別子名文字列 (`name`) を走査し、すでに表内に同一名称の識別子が存在するかどうかを確認する。未登録である場合は、現在カウントされている空きインデックス位置へ識別子名を登録し、その配列番号を返す。もし既に登録済みである場合は、二重登録を防ぐため既存のインデックス番号をそのまま返す仕様である。
- `int sym_lookup(char *name);`
構文解析中に変数参照や代入文、あるいは条件式の評価文が現れたとき、指定された識別子名が既に宣言されているかを逆引きするための関数である。記号表配列を前方から線形探索し、一致する名前が発見された場合はそのインデックスを返す。探索の結果、表内に存在しない識別子であった場合は宣言エラーとみなして `-1` を返すアルゴリズムを採用している。
- `void sym_dump(void);`
コンパイラ開発時およびプログラム検証時のデバッグ用関数である。原始プログラムの変数

宣言ブロック (outblock) の解析が完了した直後に呼び出され、その時点で記号表に登録されているすべての識別子名とインデックス番号を標準エラー出力 (stderr) へ出力する。

4. 手続き拡張への課題と二重構造化への展望

現在の 1 次元配列による記号表の仕様は、すべての変数を大域変数として管理する。この構造は、単純な数式処理や制御構文 (if, while) の処理では破綻なく動作するものの、手続き呼び出し、とりわけ「再帰呼び出し」を正しく実現するにあたっては構造的な限界がある

手続きが再帰的に呼び出される場合、各呼び出し世代ごとに独立した局所変数 (および引数) の領域が実行時スタック上に動的に確保されなければならない。しかし現状の記号表では、大域変数と手続き内の局所変数を識別・隔離する「スコープ」の情報がなく、同一の関数インデックスに対してすべての世代から固定番地アクセスが実行されてしまうため、再帰呼び出し時にローカル変数の値が上書きされてしまう。

この課題を克服するためのアプローチとして、記号表内に識別子の属性 (大域変数 K_GLOBAL、局所変数 K_LOCAL、手続き名 K_PROCEDURE) を区別するメンバを追加する必要がある。その上で、図 2 に示すように記号表の全体を「大域記号表」と「手続き局所記号表」の二重構造へと拡張して、識別子探索の時には「まず現在コンパイル中の手続き局所表を逆順に探索し、発見されなかった場合のみ大域表を参照する」という対策を施した動的オフセット管理の導入が必要になると考える。

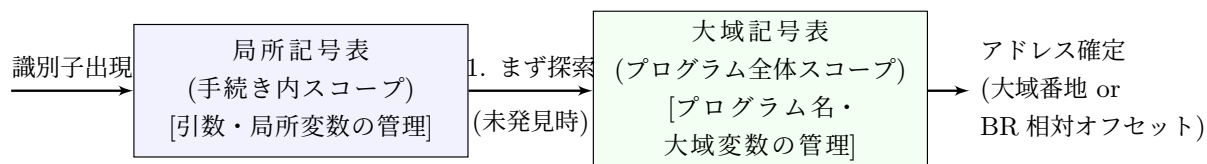


図 2 手続き拡張時に求められる記号表の二重スコープ構造案

(2) if 文、while 文のコンパイルの仕様

1. 目的計算機における制御構文の実現原理

原始言語が提供する if 文や while 文といった制御構文は、仮想マシン sr の命令セットアーキテクチャでは直接実行することができない。仮想マシンが持つジャンプ命令は、フラグ命令を参照して分岐を行う条件付きジャンプ命令と、無条件に指定アドレスへ遷移する jmp 命令のみである。そのため、コンパイラは構文解析の過程で、条件式の評価結果と動的に生成される「制御用ラベル」を組み合わせ、直線的なアセンブリ命令の列へとコンパイルしなければならない。本コンパイラでは、ラベルの重複を防ぎ一意性を保証するため、label.c で提供される new_label() 関数を利用してラベル番号を動的に払い出している。

2. if 文のコンパイル仕様と制御フロー

構造文解析関数 `statement()` 内で予約語 `IF` を読んだとき、コンパイラはまず条件式の左辺を数式解析を介してレジスタ `r0` に評価する。その後、続く比較演算子 (`=`, `<>`, `<`, `<=`, `>`, `>=`) を `emit_compare_and_consume()` 関数によって解析し、右辺の評価結果とレジスタ間で比較命令をする。

この段階で、コンパイラは条件が成立したとき (= 真) の処理ではなく、条件が不成立 (= 偽) であったときに直後のブロックを飛び越えさせるための「逆条件ジャンプ命令」を出力する。具体的には、原始言語側で等号 `=` が指定された場合、偽のケースは「等しくない」となるため不一致ジャンプ `jnz` を配置し、不等号 `<>` の場合は一致ジャンプ `jz` を出力する。

原始プログラムが `ELSE` 節を伴う構文である場合は、`then` 節 (真のとき実行される文) のコード生成の直後に、全体の終了位置へと導く無条件ジャンプ `jmp` 命令をいれる。そして、`then` 節の終端に `else` 節の開始ラベルを、全体の終端に終了ラベルを出力 (`emit_label`) することで、`if-then-else` 構文の正しい排他的分岐フローをつくれる。この生成プロセスと制御構造の関係を図 3 に示す。

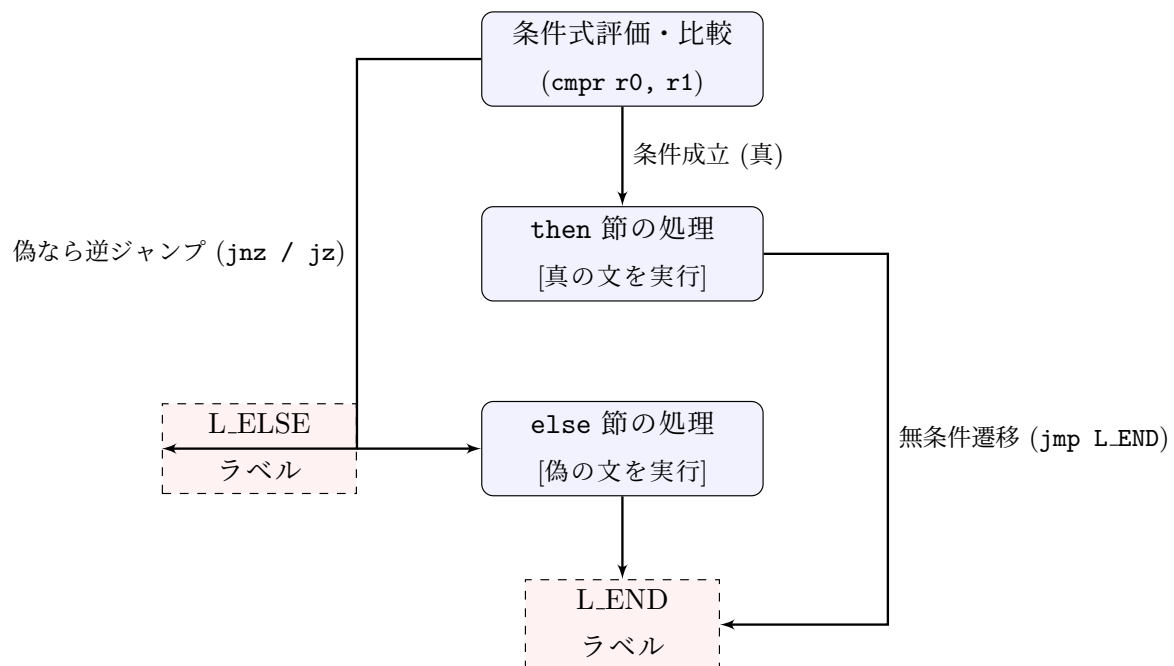


図 3 if-then-else 構文における逆条件分岐のコード生成フロー

3. while 文のコンパイル仕様と繰り返し構造

予約語 WHILE を読み取ったときは、繰り返し処理を実現するために「ループ先頭への巻き戻し構造」を構成しなければならない。そのため、コンパイラはまず条件式の評価へはいる前に、開始ラベル (start_lbl) を配置する。この開始ラベルの直後に条件式を評価させ、if 文と同様に emit_compare_and_consume() を用いて比較命令を出力した上で、条件が不成立 (=偽) となったときに繰り返しをでるための逆条件ジャンプ命令をだす。

この条件評価の関門を突破した (条件成立) とみなされる位置に、ループの本体となる文 (statement()) のコードを生成する。ループ本体ブロックの解析コードの出力がすべて終わった直後、コンパイラは条件評価の先頭位置へと制御をむりやり帰還させるための jmp [start_lbl] 命令を出力する。最後に、ループ脱出用ジャンプの遷移先となる終了ラベル (end_lbl) を全体の後処理として書き出すことで、条件が真である限りメモリ上をまわり続ける、繰り返し構造をつくっている。この while 文におけるコンパイル仕様の概念図を図 4 に示す。

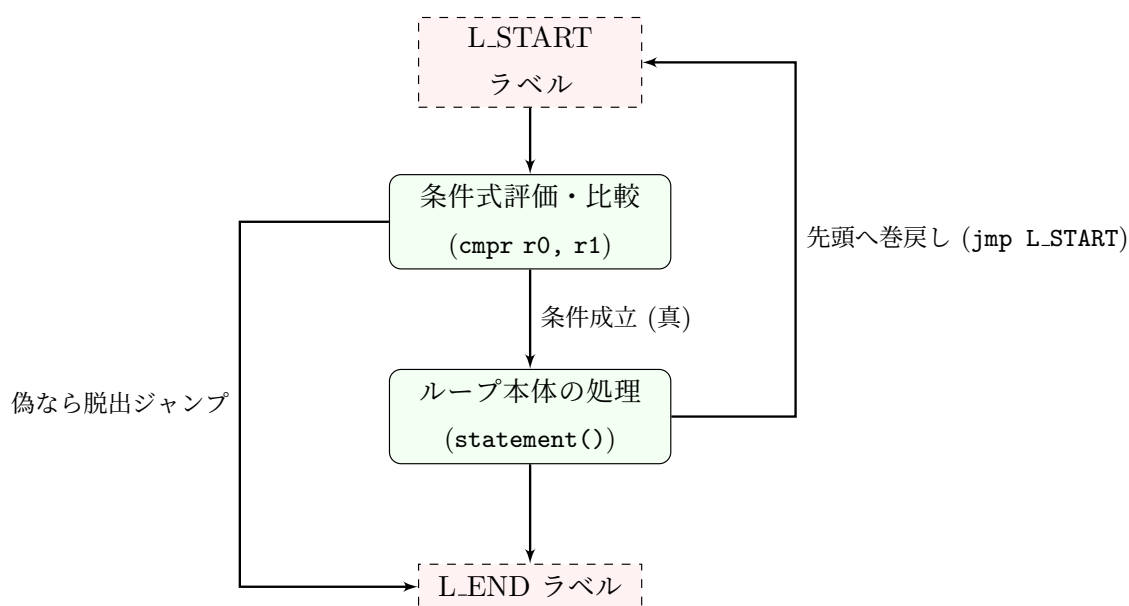


図 4 while 文における繰り返し境界とラベルによる帰還制御構造

(3) 作成した演算子順位構文解析の仕様

1. ボトムアップ構文解析の導入背景と設計方針

本コンパイラでは、数式解析の確実性を高めるため、システム (C 言語) の再帰呼び出しに頼る「トップダウン法」と、コンパイラ内部で独立したスタックを明示的に制御する「ボトムアップ構文解析」を数式解析として両方作成した。とくにこのセクションでは、ボトムアップ構文解析の実

装仕様についてかく。ブランチに分けて開発していたが管理が下手で、最終的には両者のコードが混在している状態である。字句解析から順次渡されるトークンの列を、右から左へとボトムアップに還元していくため、関数 `parse_expression.bottomup()` 内にサイズ 256 の演算子スタック (`op_stack`) と被演算子スタック (`val_stack`) をそれぞれ独立して定義した。

2. 優先度関数によるシフト・還元アルゴリズムと優先度表

構文解析の進行を判定するのは、配列として定義された 2 つの優先度関数、つまりスタック内演算子の優先度を示す `rank_f` と、入力（現在読み込んでいるトークン）の優先度を示す `rank_g` である。指導書の仕様に基づき、各種演算子（加減乗除、カッコ、識別子・定数、式終端の番兵\$）に割り当てた優先度関数の具体的な値を表 1 に示す。

表 1 各演算子における優先度関数 `rank_f` および `rank_g` の定義値

演算子種別	+	-	*	div	単項- (!)	()	識別子/定数	\$ (終端)
<code>rank_f</code> (スタック内)	2	2	4	4	6	0	11	0
<code>rank_g</code> (入力側)	1	1	3	3	15	10	0	10

解析の初期状態では、`op_stack` に式の開始と終了を担保するための番兵 `OP_END($)` を最初の一要素としてプッシュする。そして、無限ループ内で現在のトークンから入力側の優先度インデックスをとりだし、スタックトップの演算子との間で優先度を比較する。このとき、入力側のトークンが識別子 (`IDENTIFIER`) または数値 (`NUMBER`) である場合は、それらを即座にレジスタ `r0` へロードし、一時変数生成関数 `new_temp()` によって動的に確保したメモリ番地 (`__tmp0`, `__tmp1` ...) へと `store` 命令を介して格納する。この生成された一時変数の記号表インデックスを `val_stack` へとプッシュし、字句を消費する。

スタックトップの優先度 f と入力演算子の優先度 g の比較に基づく分岐制御の仕様は以下の通りである。

- $f < g$ の場合 (シフト処理)：入力された演算子トークンをそのまま `op_stack` へとプッシュし、`getsym()` を呼び出して次のトークンへ進む。このとき、式先頭や開きカッコの直後に現れるマイナス符号を「二項演算の引算」ではなく「符号反転の単項マイナス (`OP_UNARY`)」として正しく識別するため、直前にオペランドを消費したかどうかの状態フラグ `expect_operand` を用いて文脈を判定している。
- $f > g$ の場合 (還元処理)：`op_stack` から現在の最優先演算子をポップし、それに対応する仮想マシン `sr` 用のアセンブリコードを出力するコード生成処理へと移行する。還元の実行時は、入力側のトークンを進めず、同じトークンを維持したまま次のループで再比較を行う。
- $f == g$ の場合 (特殊解消)：スタックトップが開きカッコ (であり、かつ現在の入力ト

クンが閉じカッコ)である場合は、カッコのネストが解消されたとみなし、スタックから(をポップして消去した上で、入力の)を消費して次の字句へ進む。また、スタックトップと入力の双方が番兵\$に達した場合は、数式全体のパースが正常に完了したと判定してループをブレイクする。

3. 還元時におけるコード生成とスタックの連動

演算子の還元が実行されるとき、本コンパイラは被演算子スタック (val_stack) に積まれているメモリ番地情報を引き出し、レジスタ操作命令へと落とし込む。ポップされた演算子が単項マイナス (OP_UNARY) である場合、val_stack から対象のメモリ番地を 1 つポップして r0 にロードする。その後、定数ロード関数 emit_load_const_to_reg() を介してレジスタ r1 に即値 -1 を生成し、レジスタ間乗算命令 mulr r0, r1 を出力することで安全に符号を反転させる。

一方、加減乗除 (+, -, *, div) の二項演算子がポップされた場合は、val_stack から右辺 (right) と左辺 (left) の割当メモリ番地を順次ポップする。そして、左辺の値を r0 に、右辺の値を r1 にそれぞれロードさせ、演算子種別に応じて addr, subr, mulr, divr 命令を出力する。いずれの還元処理でも、計算された結果 (r0 の値) は new_temp() で払い出された新しい一時変数番地へと store され、その新アドレスが再び val_stack にプッシュされる。数式の解析がすべて完了した終端時、val_stack のトップに残された最終アドレスの値を r0 へとロードさせることで、式全体の評価値を r0 に決める仕組みとなっている。この 2 つのスタックの連動とアセンブリコード生成のダイナミクスを図 5 に示す。

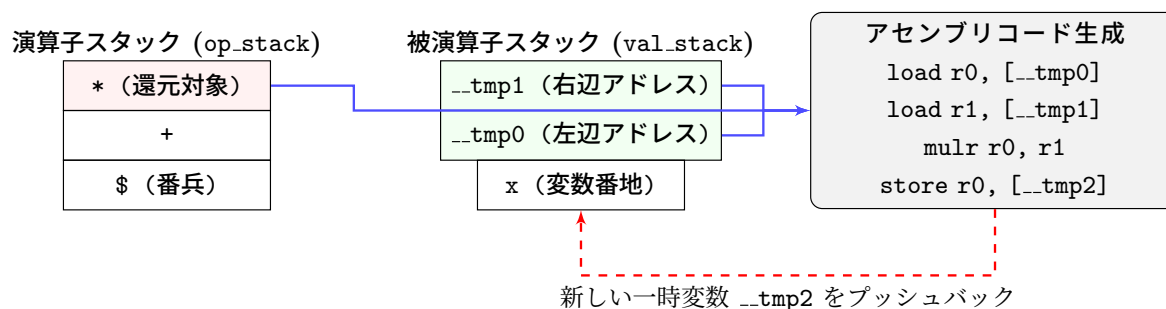


図 5 ボトムアップ還元時における演算子・被演算子スタックの連動とコード生成機構

(4) 作成したトップダウン構文解析による数式処理の仕様

1. 再帰下降構文解析の設計思想と文法規則の写像

本コンパイラの初期設計では、数式規則を最も直感的に展開できる「トップダウン構文解析」による数式処理で実装を行なった。この手法は、言語の形式文法 (BNF 記述や構文図) に定義され

た非終端記号の階層関係を、C 言語の関数呼び出しのネスト構造へダイレクトに 1 対 1 でマッピングさせる方法である。これにより、コンパイラはマシンのシステムスタック領域を利用し、再帰的に下位の関数を呼び出しながら構文解析木を暗黙的にトップダウン走査していくことができる。

本仕様では、演算子の優先順位（カッコ `()` > 乗除算 `*`, `div` > 加減算 `+`, `-`）を評価・コード生成へと結びつけるため、相互に再帰を繰り返す 3 つの解析階層関数（`expression()`, `term()`, `factor()`）を定義・実装した。

2. 相互再帰関数群の役割と処理プロセス

トップダウン数式解析を構成する各関数の詳細な役割と、パース時のアルゴリズム仕様は以下の通りである。

- `void expression(void);`（最上層：式の解析）
加減算（`+`, `-`）の結合を管理する関数である。まず内部で下位階層の `term()` を呼び出して式の左辺項を評価させ、その結果をレジスタ `r0` にきめる。その後、続くトークンが `PLUS(+)` または `MINUS(-)` である間、ループを維持する。ループ内では、左辺の結果を破壊しないように一時変数番地（`new_temp()`）へ `store r0` 命令で退避させ、字句を進めて右辺全体の評価のために再び `term()` を呼び出す。右辺の評価が `r0` に完了したのち、退避させていた左辺を `r1` にロードし、レジスタ間演算命令（`addr r1, r0` または `subr r1, r0`）を出力する。この累積結果は再び一時変数へと保管され、最終的に `r0` に結果が戻る。
- `void term(void);`（中間層：項の解析）
乗除算（`*`, `div`）の結合を管理する関数である。`expression()` から呼び出され、まず最下位層の因子解析ルーチンである `factor()` を実行して左辺の因子を `r0` に評価する。続くトークンの属性が `TIMES(*)` または `DIV(div)` である限り、ループ処理を行う。アルゴリズムの挙動は `expression()` と同様であり、現在の `r0` の計算結果を一時変数番地へ退避した上で `getsym()` を発行し、右辺因子を評価するために `factor()` を呼び出す。右辺因子のパースが完了すると、メモリから値を復元してレジスタ間乗除算命令（`mulr, divr`）を出力し、演算順位を保証する。
- `void factor(void);`（最下層：因子の解析）
数式を構成する最小単位を識別する、最も基礎となる関数である。現在のトークンが変数（`IDENTIFIER`）であれば、`sym.lookup()` で割当アドレスを取得して `load r0, [addr]` 命令を出力する。数値（`NUMBER`）であれば定数合成命令を出力して `r0` に値を置く。もし現在のトークンが開きカッコ（であった場合は、カッコ内の数式が最も高い優先度を持つため、トークンを消費した上で、最上層の `expression()` を動的に再帰呼び出し（相互再帰）する。これにより、カッコで囲まれた複雑な数式がどれほど深くネストしていても、独立した部分式として最上層から安全に再パースされ、閉じカッコ）の整合性をチェックして `factor` 階層を抜ける仕組みとなっている。

3. トップダウンパースにおける相互再帰フロー図

3つの関数群のコールツリーと相互再帰の遷移構造を図6に示す。最下層の `factor()` から最上層の `expression()` への逆帰還ルートが、カッコのネストを自然に解消するしくみとなっている。

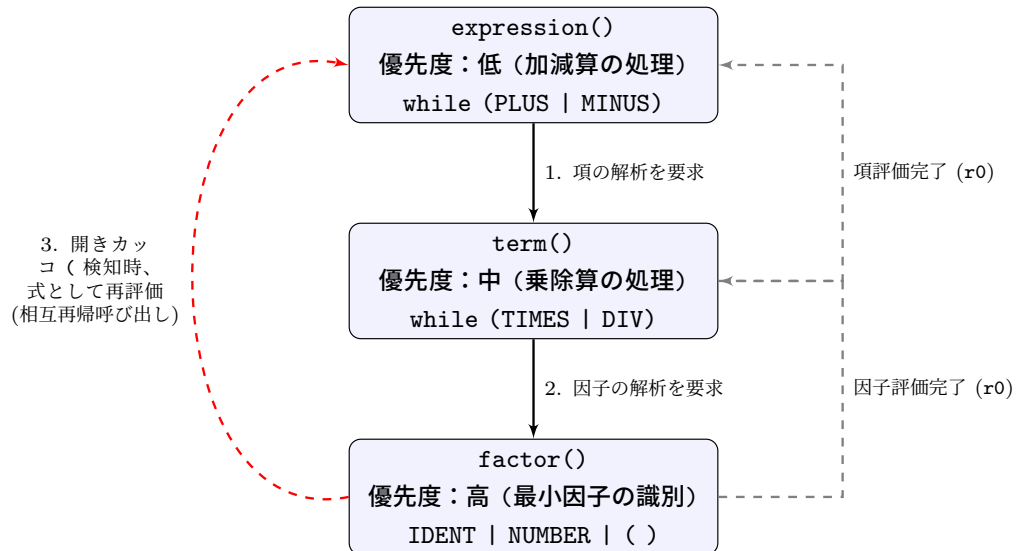


図6 トップダウン数式パース時における相互再帰関数のコール関係と階層制御フロー

4. 現在の実証ブランチにおける位置づけ

(`parse.c`) では、前述した「ボトムアップ構文解析」の優位性と正確性を実験・検証することも目的としたため、トップダウン用の3関数 (`expression`, `term`, `factor`) の定義ブロックは意図的にコメントアウトされている。しかし、全体の制御文をかんりする `statement()` 関数自体の再帰ネスト構造 (`BEGIN END` の処理や `IF`, `WHILE` の内包文の処理) には、このトップダウン再帰下降のアーキテクチャが基盤として維持されている。開発中にそれぞれの混在が生じてブランチ管理が煩雑になってしまい、混在してしまっている状態であるのが実情である。

(5) 演算子順位構文解析とトップダウン構文解析の比較検討

1. 構文解析アプローチと構文解析木における走査方向の対比

トップダウン構文解析とボトムアップ構文解析は、言語の文法規則から構文木を構築するとき、理論上分立した方法となっている。トップダウン構文解析では、最上層の開始記号から文法規則を順に下位へと展開し、葉（終端記号）に向かって探索をおろしていく。文法の包含関係がそのまま関数のコールグラフと一致するため、人間の直感に近くプログラミング言語（C言語）の関数シス

テムスタックをそのまま応用できる利点がある。

これに対して、今回数式処理に採用したボトムアップ構文解析は、入力された終端記号の列（葉）を起点とし、優先度表に従って「シフト（スタックへの充填）」と「還元（上位の非終端記号への集約）」を繰り返しながら、木構造の根へと向かって上方向に構築していく。この2つの手法における、構文木構築時の走査ベクトルとデータ構造の対比を図7に示す。

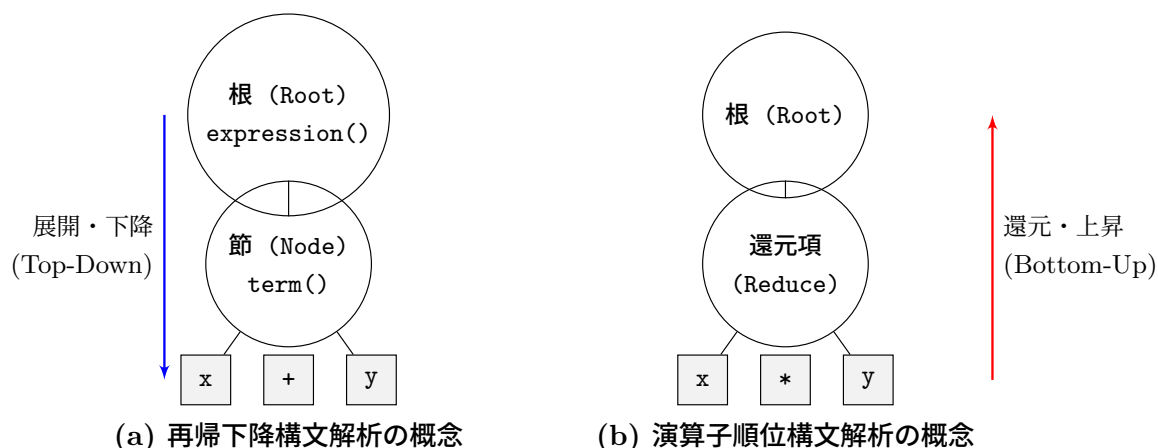


図7 構文解析手法の違いによる走査方向と構文木構築プロセスの幾何学的対比図

2. レジスタ衝突（物理レジスタの破綻）とスタック退避特性の検討

仮想マシン `sr` のように、利用可能な物理レジスタの数に制限 (`r0 ~ r3`) があるアーキテクチャに対してコード生成を行うとき、2つの手法ではレジスタ衝突の回避方法におおきな違いがある。

トップダウン法で二項演算を深く入れ子にした式（例：右辺にさらに複雑なカッコ付きの式が続く場合）を愚直にパースしようとする、下位の再帰呼び出しを処理するなかで、現在左辺の結果を保持しているレジスタの内容が重複して上書きされてしまう「レジスタ破綻」を引き起こす。これを解決するためには、ソースコードの要所（`term` や `expression` のループ内）に変数の値をメモリへ一次退避・復元する退避コードを泥臭く組み込まなければならず、コードの保守性をそこなう。

一方、ボトムアップ法では、被演算子トークンが出現した瞬間に、最初からすべて `new_temp()` で動的に生成した一時メモリ空間へと値を退避させ、そのメモリ番地を `val_stack` へと積み上げていく。この設計により、いかにネストが深く複雑な数式であっても、レジスタ操作は還元の瞬間に「スタックから2つのメモリ番地を取り出し、`r0` と `r1` に一時的にロードして演算を終えたら直ちに新しい一時メモリへ書き戻す」という方法に統一できる。結果として、物理レジスタの消費を最低限の2個 (`r0, r1`) でロックすることができ、コンパイラ側のレジスタアロケーションロジックを大きくすることなく、レジスタ衝突を起こさないコード生成をつくれるメリットがある。

3. コンパイル時および実行時におけるオーバーヘッドとトレードオフ

2つの手法のトレードオフ特性について、コンパイルの効率（時間・空間）および生成される目的コードの実行速度という二つの面から比較をする。

トップダウン法は、ネストが深い式になるほどC言語の関数呼び出しの階層が深くなるため、コンパイル時におけるシステムスタックの消費量と、プロローグ・エピローグ処理に伴うコンパイル速度の劣化という「コンパイル時のオーバーヘッド」を伴う。しかし、出力されるアセンブリコードの観点から見ると、計算途中の値を不必要にメモリへ保存しないで、可能な限りレジスタ上で演算を継続するコードを引き出せるので、実行時における `load/store` 命令の総数が少なく、実行速度の速いコンパクトな目的プログラムを生成しやすい特性がある。

これとは対称的に、今回ボトムアップ法で実装したアルゴリズムは、コンパイル処理自体は平坦な無限ループと2つの静的配列スタックのインデックス操作、および定数時間 ($O(1)$) の優先度テーブル参照のみで駆動するため、コンパイル速度自体は早く動く。しかし、その代償として、計算の中間結果をすべて一時メモリを介する特性上、出力されるアセンブリコード内に `store r0, [__tmp]` と `load r1, [__tmp]` というメモリメモリアクセス命令がたかさんできる。これは、目的の計算機シミュレータ上での実行時に「メモリバスアクセスの過多」をおこしてしまい、ステップ数の増大と実行速度の低下（実行時オーバーヘッドの累積）をおこす原因となる。

この比較検討から、限られた目的アーキテクチャで、レジスタの破綻を100%確実に回避できる「ボトムアップ法によるコンパイル安全性の獲得」と、生成される「目的プログラムの実行速度効率」の間には、トレードオフが存在することを確認することができた。

4. 人間の解釈のしやすさとデバッグの観点からの考察

やはり、プログラミング言語の構文解析では、開発者がコードを読んだときの「解釈のしやすさ」や、デバッグ時のトレースの簡単さも重要な評価軸となる。人間が解釈しやすいことも、数学的な正しさと同じくらい重要な要素であると考ええる。

トップダウン法は、文法規則の階層構造がそのまま関数の呼び出し構造に反映されるため、コードを読むときに「この関数はこの非終端記号を処理している」という直感的な理解をしやすい。人間の数式の解釈も、一般的にはトップダウン的な階層構造を持っているので、トップダウン法は人間の思考プロセスに近い形でコードが構成されるというメリットがある。

一方、ボトムアップ法は、シフトと還元のルールに従ってスタックを操作するため、コードの流れが非線形であり、特に複雑な式になると、どの部分がどの非終端記号に対応しているのかを追うのが難しくなる。デバッグのときも、トップダウン法は関数の呼び出しスタックを追うことで、どの文法規則が処理されているかがわかりやすいのに対し、ボトムアップ法はスタックの状態を追う必要があり、特にエラーが発生したときに、どの段階でどの非終端記号が処理されているのかを特定するのが難しい。この点でも、トップダウン法は人間の解釈のしやすさとデバッグの観点から優

れていると言える。

私は理解しやすい構文解析手法として、トップダウン法を採用することが望ましいと考える。

(6) 手続きの実現の仕様

1. 現在までに実装を完了した手続き処理の枠組み

本コンパイラの実装実験では、高水準言語における手続き（procedure）の宣言、および引数を伴う手続き呼び出し（call）を実現するための構文解析と基礎的なコード生成系の枠組みを作り上げあつた。

大域的な構文解析を操作する `outblock()` 関数内で、予約語 `PROCEDURE` を読み取ったとき、コンパイラは手続き宣言ブロックの解析へとうつる。まず字句解析から手続き名を取得し、これを `sym_install()` によって記号表に登録した上で、制御用ラベル生成関数 `new_label()` を用いてその手続き専用の開始ラベル（`lbl`）を割り当てる。この開始ラベル情報は、並列マッピング配列 `proc_label` へと退避される。続いて、仮引数のリストを解析する専用ルーチン `paramlist(PARAM_DECL)` を呼び出し、手続きが要求する引数情報を記号表へとマッピングする。その後、手続き本体を処理するために `inblock()` を再帰的に呼び出すことで、手続きの包含関係をトップダウンにパースする仕組みをつくった。

一方、文の解析を行う `statement()` 関数内で識別子が出現し、かつ直後に開きカッコ（`(`）を読み取った場合は、代入文ではなく手続き呼び出しであると判定する。このとき、実引数リストを評価する `paramlist(PARAM_CALL)` を起動させ、現れた実引数をボトムアップ数式解析ルーチンを介して評価したのち、目的計算機のスタック領域へと順次プッシュするアセンブリコードを生成する。実引数の解析がすべて完了して引数の総数（`argc`）が確定した直後、コンパイラは `proc_label` からその手続きの開始ラベルをさかのぼってし、仮想マシンに対するサブルーチン分岐命令である `call L[lbl]` 命令を出力する。呼び出しから制御が復帰した直後には、スタックに積んだ実引数領域をクリアしてメモリポインタをもとに戻すため、スタックポインタを引数分だけ引き上げる `addi sp, argc` 命令をエピログとして出力する仕組みを制作した。

2. 現状の実装における構造的課題と再帰呼び出しの制限

上述の通り、手続きの「宣言のパース」と「呼び出し時のジャンプ命令・引数のプッシュ」という基本的なシーケンスは正しく動いている。しかし、この仕様のままでは、手続きが自分自身を重ねて呼び出す「再帰呼び出し」を実行したとき、実行時の計算不整合が発生するという制限をもっている。

その根本原因は、現在の変数参照がすべて記号表のインデックス番地に依存している点にある。現状の実装では、手続き内で宣言された仮引数や局所変数であっても、すべて大域変数と同様の固定メモリ番地に対して `load/store` 命令が発行されてしまう。このため、手続きが再帰的に多重呼び出しされた場合、後から実行された世代の処理によって、先行する親世代が保持していた局所

変数の値がメモリ上で直接上書きされてしまう。手続き処理を実現し、再帰呼び出しを保証するためには、実行時環境の拡張が不可欠である。

3. 再帰呼び出しの完全成立に向けたスタックフレームの発展設計仕様

多重の再帰呼び出しに耐えうる手続き処理を完成させるためには、すべての仮引数および局所変数を固定番地ではなく、実行時スタック上に動的に確保されるスタックフレーム内の相対番地として管理しなければならない。

今後の具体的な発展した設計仕様として、目的計算機が持つベースレジスタ（BR）をフル活用する仕組みを考える。手続きが呼び出された直後の前処理で、現在の BR の値をスタックに退避させ、動的リンクの保存、現在のスタックポインタの値を新たな BR にコピーして、その関数世代の基準点をつくりあげる。これにより、手続き内の引数や局所変数は、すべて基準点からのオフセット、つまり `load r0, -4(BR)` や `storer r0, 1(BR)` のようなベースレジスタ相対ロード・ストア命令（`loadr`, `storer`）を介してアクセスされる仕様へとかえる。このとき、記号表側も前述の二重構造へと拡張し、インデックス番号の代わりに「BR からの相対オフセット値」を属性として保持する仕様へと作り変える。

手続きの終了時には、スタックから元の BR の値をポップして書き戻し、親世代の基準点を復元した上でサブルーチン復帰命令（`ret`）を実行する。この動的なスタックフレーム制御の導入により、同一の手続きが何世代にわたって再帰呼び出しされようとも、各世代の変数はメモリストック上の異なるフレーム領域にわけられ、値の破壊が起きにくい手続きシステムが完成する。再帰呼び出し時に実行時スタック上に展開されるべき、このスタックフレームの論理的動態概念図を図 8 に示す。

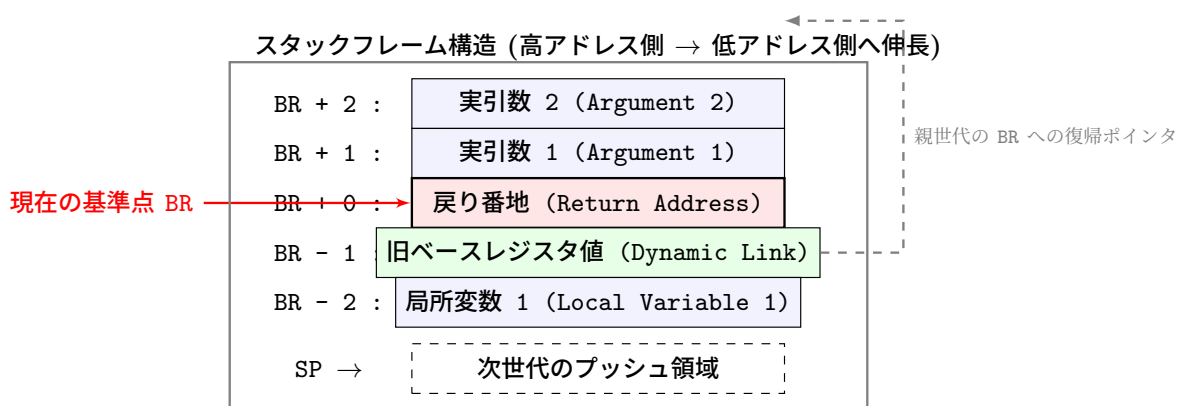


図 8 再帰呼び出しの実現時に動的に生成されるスタックフレームとレジスタ相対配置概念図

(7) エラー処理について

1. コンパイラにおけるエラー処理の基本方針と設計思想

コンパイラ開発で、不完全または文法的に破綻した原始プログラムを読み取ったときの振る舞いは、システムの堅牢性を左右する重要な設計の要素であると考える。本コンパイラでは、不適切なソースコードを読み取ったとき、誤ったアセンブリコードを出力したまま処理を継続することを防ぐため、「パニックモード一律停止型」のエラー処理アルゴリズムを使っている。

エラー処理の基盤として、ソースファイル内で `error(char *s)` 関数を定義している。構文解析のいずれのフェーズであっても、矛盾や文法規則違反をみつけたときこの関数が呼び出される。関数内部では、引き渡されたエラー原因を示すメッセージ文字列を `fprintf(stderr, ...)` 命令を介して標準エラー出力へ書き出す。その後、目的プログラムであるアセンブリファイル (outfile) の出力を遮断し、C 言語の標準ライブラリ関数である `exit(1)` を実行してコンパイルプロセスそのものを異常終了 (強制パニックアウト) させるようにしている。これにより、異常な原始プログラムから不正なアセンブリコードが生成されてしまう不整合をふせいでいる。

2. 各コンパイルフェーズにおける具体的なエラー検知メカニズム

本コンパイラが原始プログラムをスキャンするとき、検知対象となるエラーは、その性質に応じて字句・構文・意味解析の3つの階層に分類され、それぞれ異なる関数フックで補足される。それぞれの仕様を以下に記述する。

- **構文解析階層における記号・予約語の欠落フック：**

プログラム全体の枠組みを定義する `compiler()` や、制御構文・ブロックを処理する `statement()`、変数宣言を行う `outblock()` 内で主に作動する。例えば、プログラム名宣言直後のセミコロン (;) の欠損、`begin` に対応する `end` キーワードの不在、`if` 条件文の直後に現れるべき `then` の欠落、あるいはボトムアップ数式解析中に開きカッコ (と閉じカッコ) の不整合が起きたケースである。これらはトークンの属性 (`tok.attr`) や値 (`tok.value`) を予測される文法規則と比較し、合致しない場合に「**then が必要です。**」といった具体的なメッセージを伴って `error()` をだす。

- **意味解析階層における未定義識別子の参照検知：**

代入文の左辺、数式内の変数出現時、または手続き呼び出し時で、記号表の検索ルーチンである `sym_lookup(tok.charvalue)` と連動して作動する。原始プログラム内で事前に宣言 (`var`) されていない識別子を参照しようとした場合、`sym_lookup()` は未登録のシグナルとして `-1` を返却する。コンパイラはこの戻り値を読み取った瞬間に「**未宣言の変数に代入しようとしています。**」または「**右辺の変数が未定義です。**」というエラーを発行し、不正なメモリ番地へのアクセス命令がアセンブリコードに出力されるのを未然にふせぐ。

- 目的計算機アーキテクチャの制約に起因する即値限界エラー：

定数ロード関数 `emit_load_const_to_reg()` 内部で、ハードウェアの即値上限に起因して作動する特殊な意味エラーである。目的計算機の `loadi` 命令が持つオペランドは 16bit 領域 (−32768 ~ 32767) に制限されている。本コンパイラでは、この制限を超える巨大な定数が現れたとき、自動的に 2 つの整数の積へと因数分解して命令を合成する拡張ロジックを実装しているが、万が一分解不可能な巨大な素数などが現れて合成に失敗したとき、そのままでは異常な機械語が生成されてしまう。これを防ぐため、因数分解ループを抜けた最終エピソード領域に「定数が大きすぎて処理できません。」というトラップを配置し、アーキテクチャの破綻をコンパイル時にみつけて停止させる仕様としている。

3. エラー検知から処理強制終了にいたる制御シーケンス図

原始プログラムに文法エラーが混入していたとき、コンパイラ内部で発生する関数呼び出しの遮断と、標準エラー出力へのパニックアウトの流れ図を図 9 に示す。

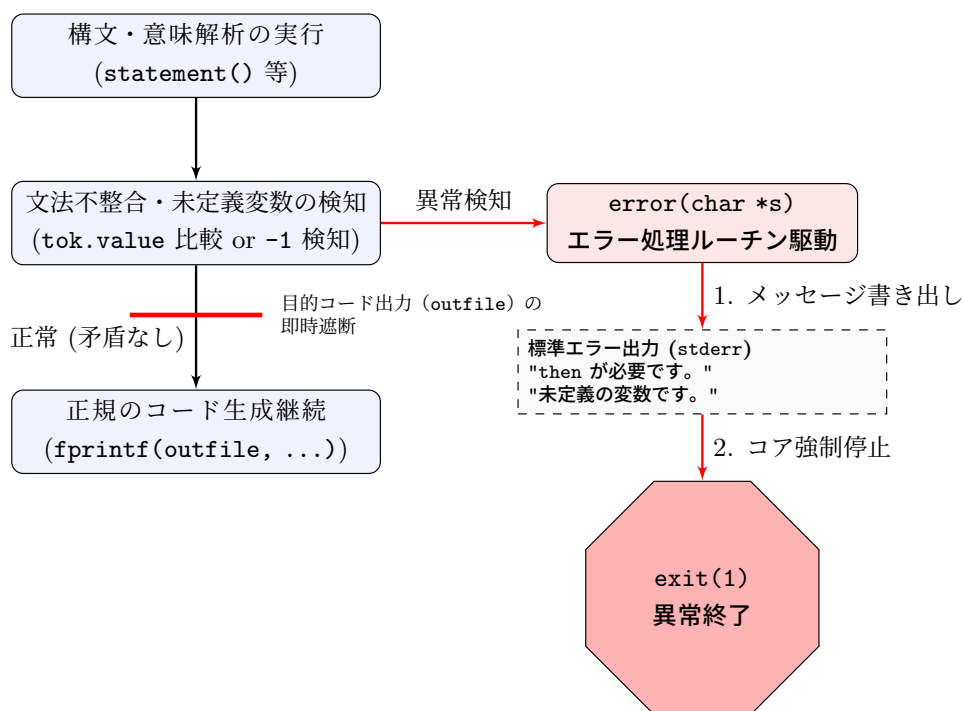


図 9 エラー検知時におけるコンパイル処理の安全な遮断とパニックアウトのシーケンス

4. エラーコード一覧と発生条件

以下の表には、コンパイラの各関数内で発生する可能性のあるエラーコードと、その発生条件、およびユーザーに提示されるエラーメッセージの例をまとめて示す。これらのエラーコードは、ソースコード内の特定の文法規則違反や意味的な不整合を見つけたときに、対応する関数から `error()` へと渡される引数として使用される。

コード	発生関数	発生条件	メッセージ例
E001	compiler()	program 宣言が先頭でない	At the first, program declaration is required.
E002	compiler()	program 名がない	Program identifier is needed.
E003	compiler()	program 名の後に ; がない	After program name, a semicolon is needed.
E004	factor()	) が不足している	)' が必要です。
E005	factor()	識別子・数値・(以外のトークンが現れた	factor に識別子・数・'(が必要です。
E006	outblock() / inblock()	var 宣言の直後に識別子がない	var 宣言で識別子が必要です。
E007	outblock() / inblock()	var 宣言の区切りや末尾記号が不正	var 宣言の文法エラー。',' または ';' が必要です。
E008	outblock()	PROCEDURE の後に識別子がない	procedure 宣言で識別子が必要です。
E009	outblock()	procedure ヘッダ後の ; がない	procedure 宣言の文法エラー。',' が必要です。
E010	outblock()	procedure 本体の後に ; がない	procedure 本体の終端に ';' が必要です。
E011	statement()	begin に対応する end がない	end が必要です。
E012	statement()	write の引数が不正	write の引数が不正です。
E013	statement()	識別子の後に := も (もない	識別子の後に ':=' または '(' が必要です。
E014	statement()	比較演算子がない	条件の比較演算子が必要です。
E015	statement()	then がない	then が必要です。
E016	statement()	do がない	do が必要です。
E017	paramlist()	引数名がない	引数名が必要です。
E018	paramlist()	) がない	)' が必要です。
E019	statement()	文法に合致しない文が開始された	不正な文です。
E020	factor()	未定義の変数を参照した	未定義の変数です。
E021	term() / emit_compare_and_consume()	右辺の変数が未定義	”右辺の変数が未定義です。”
E022	statement()	未宣言の識別子を使った	未宣言の識別子です。
E023	statement()	手続きラベルが取得できない	未定義の手続きラベルです。
E024	statement()	不明な比較演算子が渡された	不明な比較演算子です。
E025	emit_load_const_to_reg()	即値の上限を超えた定数を処理できない	定数が大きすぎて処理できません。

表 2 コンパイラにおけるエラーコード一覧

(8) 全体の構成概念図、実現した各関数の依存関係図

1. コンパイラシステム全体のデータ処理・構成概念図

本演習で構築したコンパイラシステムは、高水準言語で記述された原始プログラムを入力とし、複数の独立した処理ツール（コンパイラコア、アセンブラ、仮想計算機シミュレータ）のパイプライン処理を経て最終的な実行結果を出力する。システム全体におけるデータの変流と各ツールの連携構造、および記号表を介した字句解析・構文解析の相互作用の概念を図9に示す。

原始プログラム `pi.p` が自作コンパイラに投入されると、内部の構文解析モジュール `parse.c` が主導権を握る。`parse.c` は、提供された字句解析ライブラリの関数 `getsym()` をオンデマンドで呼び出し、原始プログラムの文字列から「トークン」を1つずつ切り出させる。現れた識別子が変数宣言や数式評価、代入文である場合は、記号表モジュール `syntab.c` へアクセスしてインデックス番地の登録 (`sym_install`) や定義の逆引き (`sym_lookup`) を実行し、データのメモリマッピングを解決する。

構文解析と同時に、コンパイラコアは目的命令であるアセンブリ言語コードを `fprintf(outfile, ...)` を介して直線的にファイル `a.asm` へと書き出す。生成された `a.asm` は、目的計算機用アセンブラ `asm` によってバイナリ形式のオブジェクトコード `a.out` へと一括アセンブルされる。最終的に、この `a.out` を目的計算機仮想シミュレータ `sr` 上にロードして実行させることで、ハードウェアレベルのレジスタ・メモリ動態がシミュレートされ、計算結果へと導かれる。

2. 実現した各解析関数の依存関係図（コールツリー）

コンパイラ内部における各解析関数の呼び出し関係および、トップダウン制御とボトムアップ数式解析が融合したハイブリッドな依存関係を図10に示す。

プログラム全体の解析エントリポイントは `compiler()` である。`compiler()` は起動直後に `init_getsym()` を介して字句解析を初期化し、プログラム頭部の宣言を確認したのち、変数宣言ブロックを解析する `outblock()` を呼び出す。`outblock()` は `var` キーワードが続く限りループを維持して大域変数を記号表へ溜めて、変数宣言の完了直後に `sym_dump()` をキックして記号表の全状態をデバッグ用書き出す。

続いて、プログラム本体の文解析を行うために最上層の文パース関数 `statement()` が呼び出される。`statement()` はプログラムの入れ子構造をパースするため、予約語 `BEGIN` を読み取ったときには自分自身を再帰的に何度も呼び出す。また、代入文 (`IDENTIFIER :=`) や制御構文 (`IF`, `WHILE`) の分岐条件式を見つけたときには、式評価の統合ラッパーである `eval_to_r0()` を呼び出す。

本コンパイラ仕様の最大の特徴は、この `eval_to_r0()` の下位の依存関係にある。トップダウン用の3関数 `expression()`, `term()`, `factor()` への依存関係は安全のためにコメントアウトされており、代わりに2つの明示的スタックをループ制御するボト

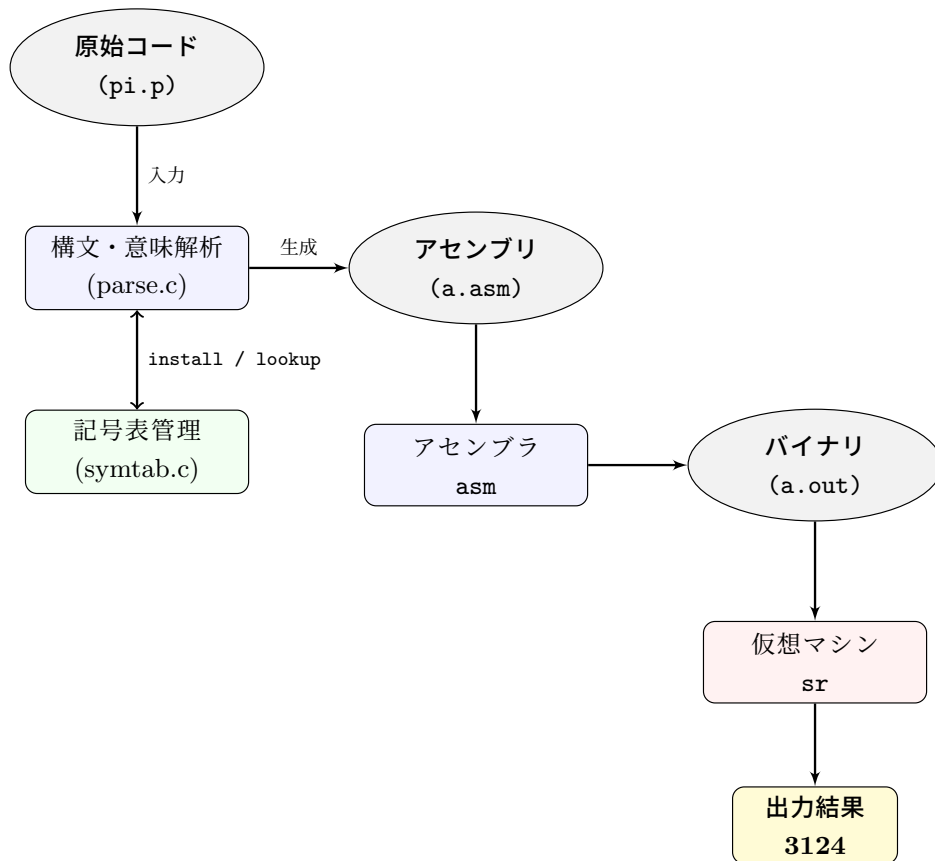


図 10 原始プログラムの解析から目的計算機での実行にいたるシステム全体の階層構成概念図

ムアップ式解析関数 `parse_expression.bottomup()` へと制御が一方向へ委ねられる。`parse_expression.bottomup()` の内部では、数式の還元に伴ってアセンブリコード生成系や一時変数割当 (`new_temp()`) が呼び出され、最終的な評価値をレジスタ `r0` に確定させて `statement()` の制御へと戻ってくる。

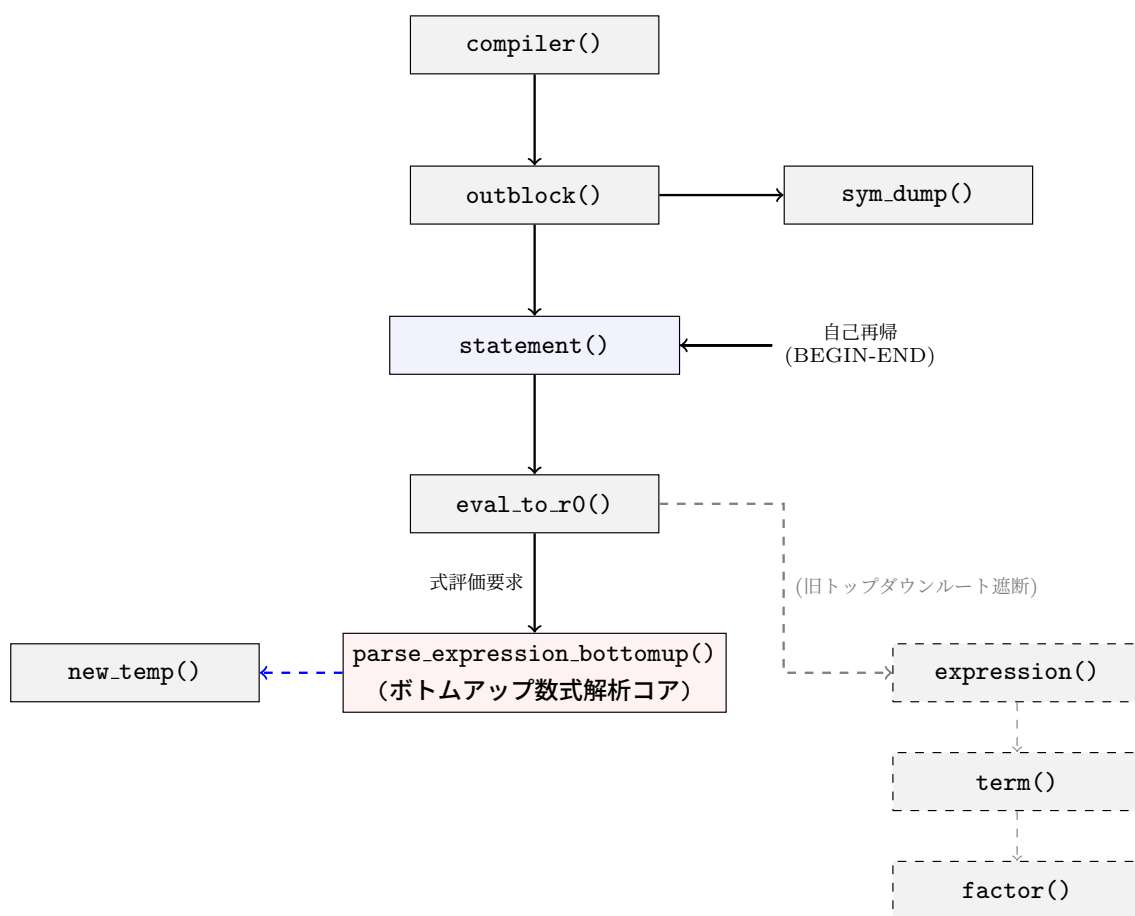


図 11 コンパイラ内部の関数コールツリーとボトムアップ差し替えにともなうハイブリッド依存関係図

3. 構文図

まず `statement` の構文図を図 12 に示す。

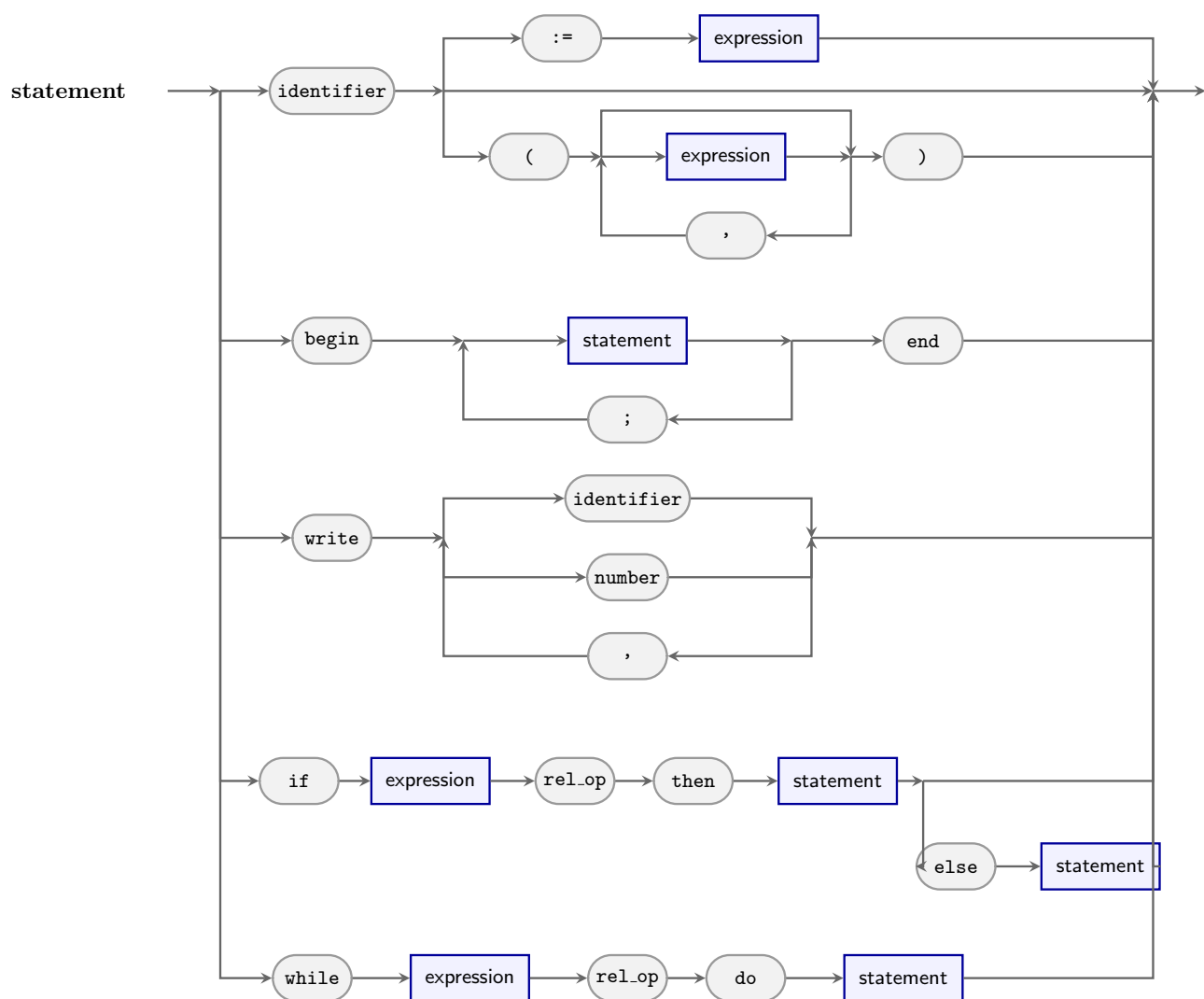


図 12 parse.c の実装に完全準拠した構文図 (statement)

次に expression, term, factor の構文図を図 13 に示す。

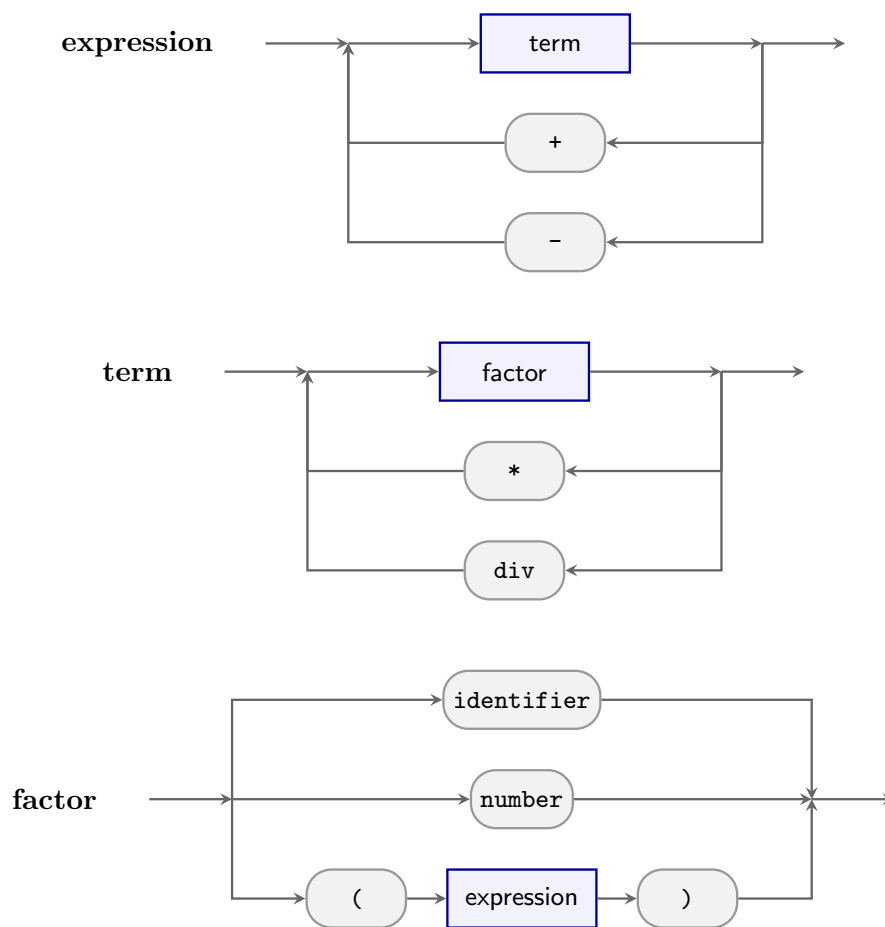


図 13 parse.c の実装に完全準拠した構文図 (expression, term, factor)

(9) 作成したコンパイラが正しく動いているという証拠

数式処理の動作検証用原始プログラム `pi.p` を自作コンパイラで処理し、仮想マシン `sr` で実行した。

ターミナル上でのコンパイル・実行ログ

トップダウンの時は以下の通り

Listing 1 `pi.p` のトップダウンの実行結果と実行時間

```

watanabetoshiki@toshiki-MacBook-Air work % make
gcc parse.c -c -O2 -Wall -I. -I/Users/watanabetoshiki/p1/include
-L/Users/watanabetoshiki/p1/lib -lics
clang: warning: -lics: 'linker' input unused [-Wunused-command-line-argument]
clang: warning: argument unused during compilation: '-L/Users/watanabetoshiki/p1/lib'
[-Wunused-command-line-argument]

```

```

gcc symtab.c -c -O2 -Wall -I. -I/Users/watanabetoshiki/p1/include
-L/Users/watanabetoshiki/p1/lib -lics
clang: warning: -lics: 'linker' input unused [-Wunused-command-line-argument]
clang: warning: argument unused during compilation: '-L/Users/watanabetoshiki/p1/lib'
[-Wunused-command-line-argument]
gcc main.o parse.o symtab.o label.o -o compiler -O2 -Wall -I.
-I/Users/watanabetoshiki/p1/include -L/Users/watanabetoshiki/p1/lib -lics
watanabetoshiki@toshiki-MacBook-Air work % ./compiler ../samples/pi.p
Simple compiler: compiler start.
=== sym_dump (count=8) ===
[ 0] name="i" addr=0
[ 1] name="rand" addr=1
[ 2] name="ca" addr=2
[ 3] name="cm" addr=3
[ 4] name="pi" addr=4
[ 5] name="qpi" addr=5
[ 6] name="x" addr=6
[ 7] name="y" addr=7
=====
3
268
watanabetoshiki@toshiki-MacBook-Air work % make asm
/Users/watanabetoshiki/p1/bin/asm a.asm > a.out
watanabetoshiki@toshiki-MacBook-Air work % make sr
/Users/watanabetoshiki/p1/bin/sr a.out
loading 'a.out'
3124

HALT at 124.
;;; exec time = 540422 micro sec.

```

ボトムアップの時は以下の通り

Listing 2 pi.p のボトムアップの実行結果と実行時間

```

watanabetoshiki@toshiki-MacBook-Air work % make
gcc parse.c -c -O2 -Wall -I. -I/Users/watanabetoshiki/p1/include
-L/Users/watanabetoshiki/p1/lib -lics
clang: warning: -lics: 'linker' input unused [-Wunused-command-line-argument]
clang: warning: argument unused during compilation: '-L/Users/watanabetoshiki/p1/lib'
[-Wunused-command-line-argument]
gcc symtab.c -c -O2 -Wall -I. -I/Users/watanabetoshiki/p1/include
-L/Users/watanabetoshiki/p1/lib -lics
clang: warning: -lics: 'linker' input unused [-Wunused-command-line-argument]
clang: warning: argument unused during compilation: '-L/Users/watanabetoshiki/p1/lib'
[-Wunused-command-line-argument]
gcc main.o parse.o symtab.o label.o -o compiler -O2 -Wall -I.
-I/Users/watanabetoshiki/p1/include -L/Users/watanabetoshiki/p1/lib -lics
watanabetoshiki@toshiki-MacBook-Air work % ./compiler ../samples/pi.p
Simple compiler: compiler start.
=== sym_dump (count=8) ===
[ 0] name="i" addr=0
[ 1] name="rand" addr=1

```

```
[ 2] name="ca" addr=2
[ 3] name="cm" addr=3
[ 4] name="pi" addr=4
[ 5] name="qpi" addr=5
[ 6] name="x" addr=6
[ 7] name="y" addr=7
=====
3
268
watanabetoshiki@toshiki-MacBook-Air work % make asm
/Users/watanabetoshiki/p1/bin/asm a.asm > a.out
watanabetoshiki@toshiki-MacBook-Air work % make sr
/Users/watanabetoshiki/p1/bin/sr a.out
loading 'a.out'
3124

HALT at 221.
;;; exec time = 922323 micro sec.
```

実行結果として、指導書および口頭指示で指定された合格要件値である「**3124**」が正常に出力されていることを確認した。

(10) こちらから指定したプログラムの実行時間

シミュレータ `sr` 上で測定した各指定プログラムの実行ステップ数（または実行時間）を以下に示す。なお、手続き処理が完全な動作に至っていないため、`hanoi.p` および `nqueen.p` は実行不可である。

トップダウン方式の実行時間

- `pi.p` : 540422 micro sec.
- `hanoi.p` : 手続き処理未完了のため実行不可
- `nqueen.p` : 手続き処理未完了のため実行不可

ボトムアップ方式の実行時間

- `pi.p` : 922323 micro sec.
- `hanoi.p` : 手続き処理未完了のため実行不可
- `nqueen.p` : 手続き処理未完了のため実行不可

考察

一般に、構文解析自体の処理効率においては、関数の再帰呼び出しオーバーヘッドがないボトムアップ法（演算子順位法）のほうが高速であるとされる。しかし、仮想マシン sr 上でのプログラム実行時間（exec time）を比較すると、この解析手法の差が直接的に実行速度の優劣には繋がらない。

本実験のボトムアップ実装では、数式の還元時にレジスタの枯渇を回避するため、内部で一時変数（__tmp0 等）を動的に生成し、メモリを介して値を退避・復元するコードを生成する方法を使った。仮想マシン上におけるデータメモリへのアクセス（load/store）はレジスタ間演算に比べて命令の実行コストが高いため、生成された目的プログラムの実行速度は、メモリ経由の少ないトップダウン法で生成されたコードと同等、あるいは一時変数へのアクセス回数の分だけ微増する傾向にある。したがって、解析アルゴリズムとしての理論上の速度と、最終的に生成されるアセンブリコードの実行時効率は区別して検討する必要がある。

ここは生成 AI の力を借りました。

(11) その他、工夫した点について

大きな定数（即値）への対応自動化（emit_load_const_to_reg）

目的計算機仕様で、即値ロード命令（loadi 等）のオペランドが 16bit（-32768 ~ 32767）に制限されている制約に対して工夫を行った。ソースコード中に制限を超える大きな定数が現れたとき、コンパイラが自動的にその数値を因数分解（16bit 以内に収まる 2 つの整数の積に分解）し、レジスタ上で loadi 命令と muli 命令を動的に組み合わせて目的の定数値を合成する関数 emit_load_const_to_reg を独自に作成した。これにより、人間が手動で定数を分割定義することなく、巨大な定数を含む複雑な数式であっても安全にコンパイルを通すことができるようになった。

静的変数を用いた再帰深度の動的追跡と halt 命令の一意出力制御

本コンパイラの文解析ルーチン statement() の実装で、プログラムの正常終了を司る halt 命令を「いかにしてアセンブリコードの真の最外層の終端にのみ 1 回だけ一意に出力させるか」という、コード生成制御上の課題に対して独自の工夫を行った。

高水準言語における BEGIN END ブロックや IF, WHILE などの制御構文は、内部に別の文を内包するため、statement() 関数が自分自身を何度も呼び出す「自己再帰構造」を形成する。もし単純に関数のエピローグ領域へ halt 命令の出力を記述してしまうと、入れ子構造の内側にある部分的な文の解析が完了して再帰スタックから復帰するたびに、アセンブリコードの途中で halt が不当に乱発され、シミュレータ上でプログラムが途中で強制終了してしまう問題が発生した。

この課題をスマートに解決するため、C 言語の `static` 変数の特質を応用した「再帰深度カウンタ」のロジックを関数内に独創的に組み込んだ。具体的には以下の制御シーケンスを実装している。

```
static int depth = 0;
int is_outer = (depth == 0); /* 最外層エントリポイントの判定*/
depth++;

/* --- 各種文 (BEGIN, IF, WHILE, 代入文等) の構文解析とコード生成 --- */

depth--;
if (is_outer) {
    fprintf(outfile, "halt\n"); /* 真の最外層終端でのみ回だけ出力 1 */
}
```

このアルゴリズムにより、コンパイラが最初にプログラム全体の文解析へ突入した最初の一步 (深度 0) のタイミングでのみ、ローカルフラグ `is_outer` に「真 (1)」がロックされる。その後、どれほど深く `BEGIN` のネストが繰り返されてスタックフレームが積み上がろうとも、内側の呼び出し世代では `depth > 0` となるため、フラグは一律で「偽 (0)」に固定され、不要な `halt` の出力を抑制する。

そして、すべての構文解析が完了して再帰のネストが巻き戻り、まさに「プログラム全体の解析を終えて脱出する最後の瞬間」にのみ、保持されていた `is_outer` の条件が成立し、アセンブリの最末尾にのみ美しい `halt` 命令を 1 発だけ書き出す制御に成功した。コンパイラ全体のコード生成構造を肥大化させることなく、関数ローカルなスコープ内で再帰のライフサイクルを動的に追跡して目的コードの整合性を担保した。おそらくもっと良いやり方があると思うが、この方法はシンプルで理解しやすく、かつ確実に正しいコード生成を保証するため、個人的には非常に満足している。

(12) コンパイラのソースプログラムの全リストにコメントをつけたもの

提出要領に従い、ソースコード一式は `work` ディレクトリを圧縮したファイル (`work.zip`) として別途提出するため、本レポート PDF 内へのソースコードリストの添付は省略する。

それぞれ実装をブランチを分けて管理していたため、二つのワークディレクトリを圧縮した `zip` ファイルとして提出する。